EXAMEN PARCIAL NRO 2

Dashboard – ***"WineLytics-V2"***

Repositorio: https://github.com/DarkosCode/WineLytics-V2

Programación Orientada al análisis de Datos

Tecnicatura Superior en Ciencia de Datos e IA

Franco Ciccarelli – 2do 4ta

Instituto Superior Manuel Belgrano — Sede Godoy Cruz (Mendoza)
Docente: Juan Caballero Gallar · Ciclo Lectivo: 2026 · Fecha: 30/10/2026

PROYECTO

I. Resumen Ejecutivo

- **Problema de Negocio:** Una bodega necesita controlar el inventario de sus diferentes líneas de vino (*Malbec, Cabernet Sauvignon, Tempranillo y Torrontés*, en sus categorías *Joven, Reserva y Gran Reserva*) y sus canales de distribución (*Mercado Interno vs. Exportación*). En esta segunda etapa del proyecto, además de *controlar el stock* (no quedarse sin producto), analizar qué varietales se venden más en cada estación del año y comparar los canales de venta mes a mes, se sumó un **backend de análisis estadístico y aprendizaje profundo** para medir cómo influyen el precio y la estación sobre las ventas y *pronosticar cuántas botellas se van a demandar* en cada segmento.
- **Público Objetivo:** Gerente de Operaciones de la bodega, Director Comercial de la bodega y Jefe Comercial/Logística: cúpula dirigencial de la bodega que toma decisiones de stock, precios y producción.

II. Stack Tecnológico (Lista de Herramientas)

- **Python (3.12)** y sus librerías para el procesamiento de datos, el análisis estadístico, la red neuronal y los gráficos:
  - *Pandas (3.0.3)* para la lectura de los CSV, limpieza de nulos y duplicados, creación de columnas y cruce de datos (`groupby` y `merge`).
  - *NumPy (2.4.6)* para la simulación reproducible con semilla (`seed=42`) y el cálculo de estadísticos descriptivos (media, mediana, desvío estándar, mínimo y máximo).
  - *SciPy (1.18.1)* (`scipy.stats`) para calcular la correlación lineal de Pearson (`pearsonr`) y el test ANOVA de una vía (`f_oneway`) entre las estaciones.
  - *Statsmodels (0.15.0)* para ajustar el modelo de Regresión Lineal Múltiple por Mínimos Cuadrados Ordinarios (**OLS**) y obtener sus coeficientes, *p*-valores, R^2, AIC y BIC.
  - *Scikit-Learn (1.9.1)* para separar los datos en entrenamiento y prueba (`train_test_split` 80/20), escalar las entradas (`StandardScaler`) y calcular las métricas (R^2, MAE y RMSE).
  - *TensorFlow (2.21.0) y Keras (3.15.1)* para armar y entrenar la Red Neuronal Densa (`Dense 32 → 16 → 1`) de Aprendizaje Profundo.
  - *Matplotlib (3.10.9), Seaborn (0.13.2) y Plotly (6.8.0)* para los gráficos y el armado del dashboard interactivo unificado en HTML.
- **Venv** para la creación de un entorno virtual para trabajar más limpio y aislado con las dependencias.
- **Pip** para gestionar los paquetes e instalar lo necesario listado en el archivo `requirements.txt`.
- **IA (Gemini / Claude)** como apoyo para la generación de datos sintéticos, consultas sobre los modelos y redacción de guías.
- **GitHub** para el trabajo con distintas ramas, su fusión y la subida del proyecto al repositorio público.

- **Visual Studio Code** y **Windows Subsystem For Linux (WSL)** para el desarrollo del código y la ejecución por consola.

III. Guía de instalación

Esta guía de instalación está pensada para seguir paso por paso todo lo necesario para clonar y ejecutar el proyecto en cualquier máquina local. También se encuentra disponible en el archivo *README.md* del repositorio en GitHub:

Requisitos Previos

Tener instalado:

- Python 3.11 o superior
- pip (administrador de paquetes de Python)
- Conexión a internet (para instalar dependencias y para que el dashboard cargue la librería de Plotly desde su CDN)

Guía de Instalación y Uso Paso a Paso

Seguir estos pasos sencillos para clonar, configurar y ejecutar el proyecto en tu máquina local:

Paso 1: Clonar el repositorio

Abre una terminal y clona el proyecto desde el repositorio de GitHub:

```
git clone https://github.com/DarkosCode/WineLytics-V2.git
cd WineLytics-V2
```

Paso 2: Configurar el entorno virtual (Recomendado)

Para mantener limpias tus librerías y evitar conflictos, crea y activa un entorno virtual de Python:

En Linux / macOS:

```
python3 -m venv .venv
source .venv/bin/activate
```

En Windows (PowerShell):

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

Paso 3: Instalar las dependencias

Instala todas las librerías necesarias del proyecto utilizando el archivo `requirements.txt`:

```
pip install -r requirements.txt
```

Paso 4: Generar los datos de prueba (Opcional)

El sistema utiliza archivos CSV simulados para los reportes (`inventario.csv` y `ventas.csv`). Si no existen en la carpeta `data/`, `main.py` los crea automáticamente, aunque también puedes generarlos

manualmente corriendo:

```
python3 sources/generadorDeDatos.py
```

¿Qué hace esto? Crea automáticamente la carpeta `data/` en la raíz del proyecto y dentro genera dos archivos con semilla fija (42): `inventario.csv` (stock actual) y `ventas.csv` (historial de 1200 transacciones de vino).

Paso 5: Ejecutar el análisis y el dashboard

Para correr toda la suite de procesamiento, modelos y visualización, simplemente ejecuta el archivo orquestador:

```
python3 main.py
```

Flujo del programa al ejecutar main.py:

1. **Verificación de datos (`asegurarDatos`):** Si todavía no están creados los CSV en `data/`, los genera automáticamente.
2. **Procesamiento (ETL) y Modelos Analíticos (`dataProcessing.py`):** El sistema lee y limpia los CSV, corre el análisis estadístico (`backend/statsAnalysis.py`) y la red neuronal (`backend/deepLearning.py`), agrega las columnas de predicción y cruza las ventas con el stock físico.
3. **Gráficos de Matplotlib y Seaborn:** Guarda automáticamente en disco el mapa de calor (`visuals/estacionalidadDeVentas.png`) y el gráfico de barras mensual (`visuals/UnidadesXCanalyMes.png`) sin bloquear la ejecución.
4. **Dashboard Interactivo (`visuals/reporte_interactivo.html`):** Genera un único archivo HTML que integra la barra de filtros, las 6 tarjetas de KPIs, los gráficos interactivos de Plotly, los dos gráficos de Matplotlib y Seaborn incrustados, las tablas estadísticas y un Simulador de Demanda, abriéndolo en tu navegador predeterminado.

DOCUMENTACION TECNICA

I. Estructura del proyecto

Toda la estructura del proyecto sigue basada en lo aprendido en las clases de programación (como en el repositorio de la clase del 28/5, separando `schema` para las columnas, `sources` para la carga/limpieza de los datasets, `data` para los archivos de datos y `visuals` para los gráficos). Para este segundo parcial se sumó la carpeta `backend/`, donde están programados el análisis estadístico (`statsAnalysis.py`) y la red neuronal de aprendizaje profundo (`deepLearning.py`), conectados directamente al flujo de `dataProcessing.py` y `main.py`:

```
WineLytics-V2/                  (Carpeta Raíz)
├── README.md                   (Archivo descriptivo de instalación, guía, etc.)
├── CHANGELOG.md                (Registro cronológico de los cambios del proyecto)
├── main.py                     (Archivo principal donde se ejecuta todo)
├── dataProcessing.py           (Orquestador del ETL, ejecución del backend y merge)
```

```
├── requirements.txt            (Dependencias del proyecto con versiones fijas)
│
├── data/                       (Datos CSV generados automáticamente al correr main.py)
│   ├── ventas.csv
│   └── inventario.csv
│
├── schema/                     (Definición de nombres de columnas y estación)
│   ├── ventas.py
│   └── inventario.py
│
├── sources/                    (Generador de datos y limpieza de los CSV)
│   ├── generadorDeDatos.py
│   ├── ventasCsv.py
│   └── inventarioCsv.py
│
├── backend/                    (Análisis estadístico y Aprendizaje Profundo)
│   ├── statsAnalysis.py
│   └── deepLearning.py
│
└── visuals/                    (Módulos de gráficos y dashboard HTML final)
    ├── matplotUtils.py
    ├── seabornUtils.py
    ├── plotlyUtils.py
    ├── estacionalidadDeVentas.png
    ├── UnidadesXCanalyMes.png
    └── reporte_interactivo.html
```

II. Backend (Diagrama pipeline de datos)

El pipeline de datos conecta desde la generación y lectura de los CSV hasta los modelos del backend y la consolidación final que alimenta al dashboard interactivo:

1. Capa de Origen y Esquemas (`main.py → sources/generadorDeDatos.py + schema/`):
Verifica si existen los archivos en `data/` y los genera con semilla fija (seed=42): `inventario.csv` (12 vinos) y `ventas.csv` (1.200 transacciones), usando las constantes centralizadas en `schema/inventario.py` y `schema/ventas.py`.

▼

2. Capa de Limpieza y Transformación ETL (`sources/inventarioCsv.py + sources/ventasCsv.py`):
Lee los CSV con Pandas, renombra columnas, limpia nulos (`fillna(0)`), elimina duplicados (`drop_duplicates`), filtra valores mayores a 0 y agrega `Reposicion Necesaria` ("SI"/"NO"), `Ingreso Total`, `Mes` (en español) y `Estacion`.

▼

3. Capa Analítica y Predictiva (`backend/statsAnalysis.py + backend/deepLearning.py`):
• **Estadística (`statsAnalysis.py`):** Calcula media, mediana, desvío, correlación de Pearson y test ANOVA con `NumPy/SciPy`, y ajusta una Regresión Lineal Múltiple OLS con `Statsmodels`, sumando la columna `Prediccion_Ventas`.
• **Aprendizaje Profundo (`deepLearning.py`):** Arma las variables *dummies*, separa 80% entrenamiento / 20% prueba, escala con `StandardScaler` y entrena la Red Neuronal `Dense(32, relu) → Dense(16, relu) → Dense(1)` con `Keras/TensorFlow`, sumando la columna `Prediccion_DL`.

▼

4. Capa de Consolidación y Visualización (`dataProcessing.py → visuals/ → main.py`):
Agrupa las ventas y predicciones por `[Varietal, Linea]`, las cruza con el inventario mediante `pd.merge(..., how="inner")`, exporta los PNG de Matplotlib y Seaborn e incrusta todo en `visuals/reporte_interactivo.html`.

III. Modelo analítico (Explicación)

Para sumarle análisis de datos real al dashboard del primer parcial, en la carpeta `backend/` se programaron tres partes que trabajan sobre las 1.200 ventas:

1. Estadística Descriptiva y Pruebas de Hipótesis (NumPy y SciPy)

- **Métricas descriptivas (`numpy`):** Sobre las 1.200 transacciones se calcula la media de unidades vendidas (**195,96 botellas** por venta, mediana de 195,00, desvío estándar de 63,13 y rango de 48 a 393 botellas), el precio unitario medio (**$20,56 USD**) y el ingreso promedio por venta (**$3.472,68 USD**).
- **Correlación de Pearson (`scipy.stats.pearsonr`):** Se midió la relación entre `Precio Unitario` y `Unidades Vendidas`, obteniendo **r = -0,7324** (p-valor = 3,06 × 10^{-202}). Al ser negativo y cercano a -1, demuestra que cuando sube el precio del vino, baja la cantidad de botellas compradas por pedido.

- **Test ANOVA de una vía (`scipy.stats.f_oneway`):** Se compararon las unidades vendidas entre las 4 estaciones del año, dando **F = 43,52** (p-valor = 1,08 × 10^{-26}). Esto confirma estadísticamente que las ventas cambian de verdad según la estación del año.

2. Modelo Estadístico: Regresión Lineal Múltiple OLS (Statsmodels)

En `backend/statsAnalysis.py` se ajustó una regresión lineal OLS para estimar las `Unidades Vendidas` a partir del `Precio Unitario` y de las variables categóricas convertidas a columnas de 0 y 1 (`pd.get_dummies(..., drop_first=True)`): `Canal de Venta`, `Varietal` y `Estacion`.

Detalle importante sobre la columna "Linea": Al principio, si se incluían juntas `Linea` y `Precio Unitario`, Statsmodels avisaba que había *multicolinealidad* porque el precio ya depende casi al 100% de la línea (*Joven* cuesta ~$8–12, *Reserva* ~$15–22 y *Gran Reserva* ~$30–45). Al sacar `Linea` y dejar `Precio Unitario`, el modelo quedó limpio (sin advertencias): parte de una constante base de **310,53 botellas** y muestra que por cada dólar que sube el precio, las ventas bajan en promedio **-3,91 botellas**.

3. Aprendizaje Profundo: Red Neuronal Densa (Scikit-Learn + Keras / TensorFlow)

En `backend/deepLearning.py` se armó una Red Neuronal Densa secuencial con `Keras`: una capa de entrada con las 8 variables escaladas con `StandardScaler`, **dos capas ocultas (`Dense` de 32 y 16 neuronas con activación `relu`)** y **una capa de salida (`Dense` de 1 neurona con activación `linear`)**. Se entrenó durante 50 épocas con el optimizador `Adam(learning_rate=0.01)` minimizando el error cuadrático medio (`mse`).

Para comparar los dos modelos de forma justa, ambos se evaluaron sobre el mismo **20% de datos de prueba (`train_test_split` con semilla 42)**:

Métrica (evaluada en el 20% de prueba)	Regresión Lineal OLS (Statsmodels)	Red Neuronal Densa (Keras / TF)	Diferencia / Mejora
R^2 (Coeficiente de Determinación)	0,6739 (67,4%)	**0,8734 (87,3%)**	+19,95 puntos de R^2
MAE (Error Absoluto Medio)	27,62 botellas	**17,27 botellas**	Baja el error un 37,5%
RMSE (Raíz del Error Cuadrático Medio)	34,97 botellas	**21,79 botellas**	Baja el RMSE un 37,7%

¿Por qué la Red Neuronal le gana por tanta diferencia al OLS? Porque la regresión lineal OLS suma cada variable por separado (suma un valor fijo por ser *Torrontés* y otro por ser *Verano*), sin enterarse de que el vino blanco *Torrontés* sube fuerte específicamente en verano y que los tintos (*Malbec* y *Cabernet*) suben fuerte en invierno. En cambio, las capas ocultas con activación no lineal `ReLU` de la Red Neuronal aprenden solas esa combinación `Varietal × Estación`, logrando un **87,3% de R^2**.

IV. Guía de Estilo

Para este documento se mantuvo el mismo estilo del primer parcial: títulos principales en color borravino, subtítulos con números romanos, apartados en azul y tablas con encabezado oscuro para que sea fácil de leer.

En cuanto al dashboard (`visuals/reporte_interactivo.html`) y sus gráficos, en esta segunda entrega se unificaron los colores de las tres librerías (Matplotlib, Seaborn y Plotly) bajo una misma paleta temática de **"Bodega / Vino"**:

Paleta de colores

- **Bordó Reserva (#722F37):** color principal del encabezado del dashboard, títulos de los gráficos, barras de *Venta Real*, serie de *Mercado Interno* en Seaborn y bordes de tarjetas.
- **Dorado Crianza (#C59B27):** color de contraste usado para el canal *Exportación* en Seaborn, la serie de *Deep Learning (Red Densa)* en Plotly, la curva de validación y el borde del Simulador.
- **Rojo Alerta (#C0392B)** → "Requiere reposición: SI" (burbujas en alerta y tarjeta KPI de stock).
- **Verde Stock OK (#27AE60)** → "Requiere reposición: NO" (burbujas con buen nivel de stock).
- **Fondo Crema Pergamino (#FAF7F5) y Gris Pizarra (#2C3E50):** aplicados en el fondo del reporte y de los gráficos de Matplotlib y Seaborn para no cansar la vista y mantener buen contraste en textos y ejes.

Gráfico 1 — Mapa de calor (Matplotlib): utiliza el colormap secuencial `PuRd` (de lila claro a bordó oscuro) sobre fondo crema `#FAF7F5`, poniendo los números en blanco dentro de las celdas más oscuras para que se lean con claridad.

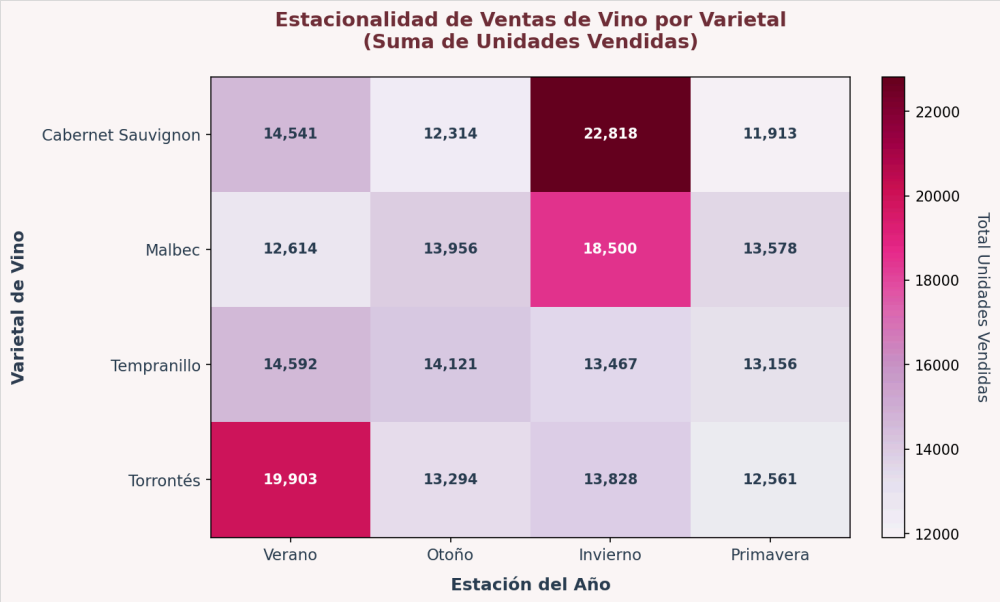
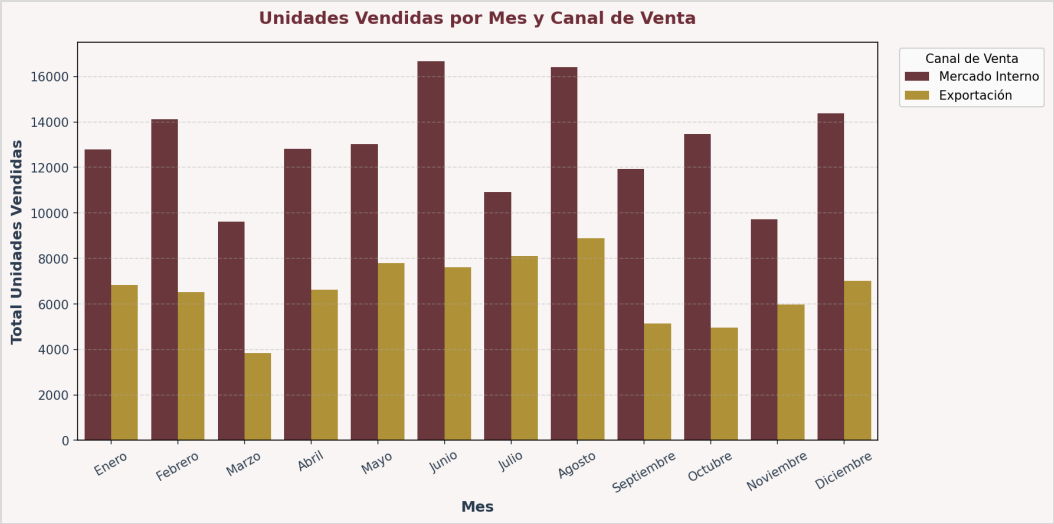


Gráfico 2 — Gráfico de barras (Seaborn): ahora utiliza los meses traducidos al español (de Enero a Diciembre) y la paleta del proyecto contrastando *Mercado Interno* (bordó `#722F37`) contra *Exportación* (dorado `#C59B27`).



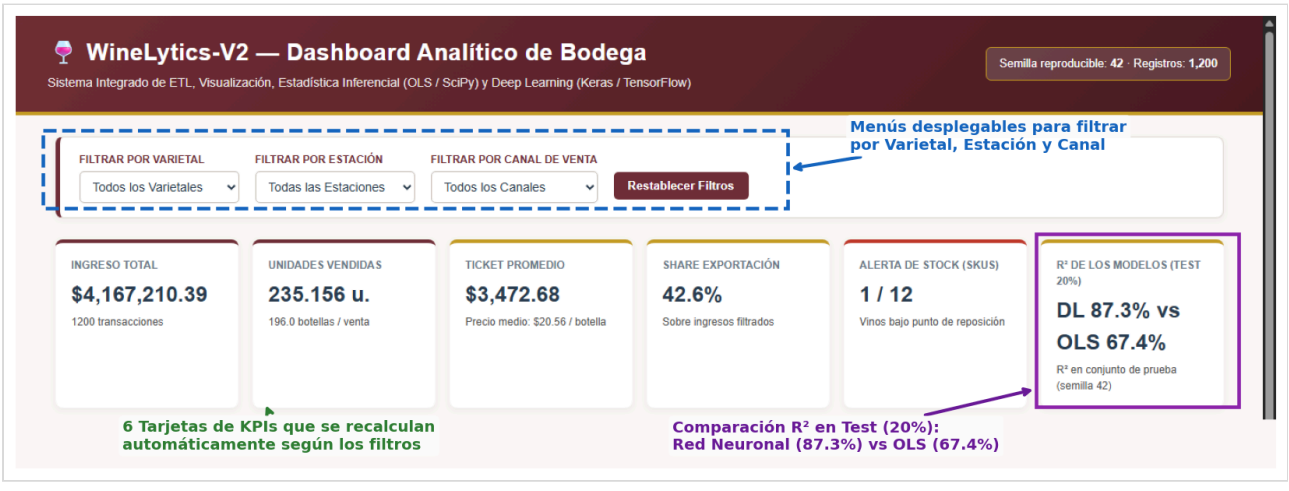
Tipografía y características generales del diseño

- **Tipografía unificada:** Se utiliza la familia `Arial, Helvetica, sans-serif` en todo el dashboard HTML y en los tres gráficos de Plotly, con títulos en negrita (`bold`) en color bordó `#722F37`.
- **Diseño integrado en una sola página:** En vez de abrir ventanas sueltas que bloquean el programa, todos los gráficos, tablas, filtros y KPIs conviven ordenados en tarjetas dentro de `visuals/reporte_interactivo.html`.
- **Tooltips y controles interactivos en Plotly:** Al pasar el mouse sobre las burbujas o barras se muestran los valores exactos y las predicciones de ambos modelos, sumando botones superiores para filtrar sin salir del gráfico.

V. Capturas de pantalla (filtros, KPI's, gráficos y simulador)

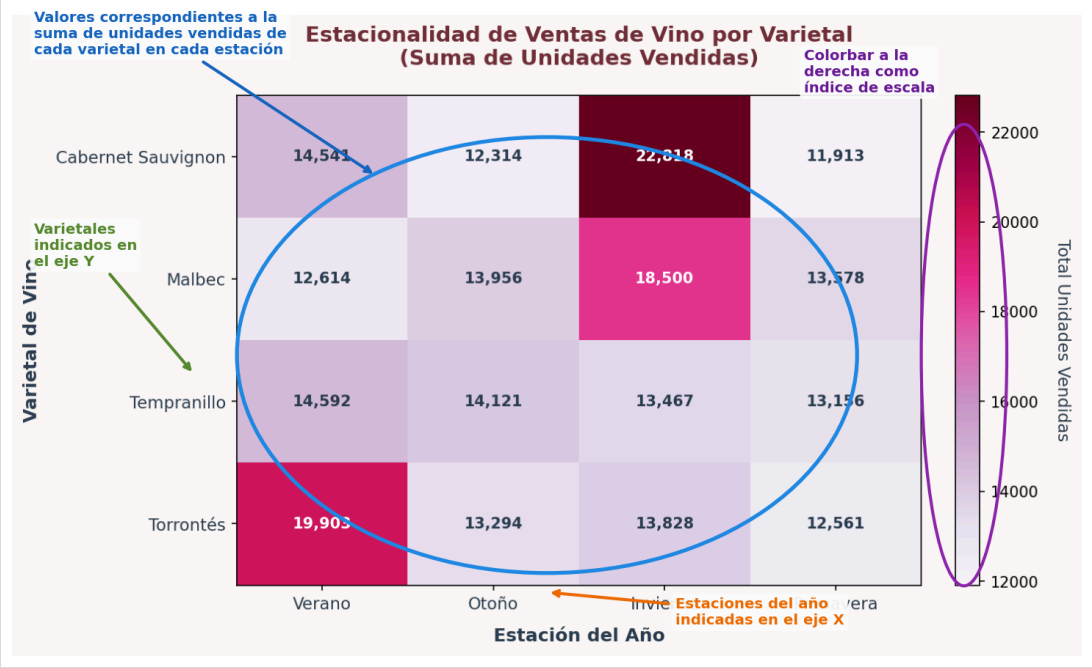
1. Encabezado, Barra de Filtros Globales y Tarjetas de KPIs

Arriba de todo en el dashboard se ubica la barra de filtros con tres desplegables (*Varietal, Estación y Canal de Venta*) y el botón *Restablecer Filtros*. Al cambiar cualquiera de esos filtros, las 6 tarjetas de KPIs que están justo debajo se recalculan al instante mostrando el Ingreso Total, las Unidades Vendidas, el Ticket Promedio, el Share de Exportación, la Alerta de Stock y la comparación del R^2 en test entre Deep Learning (87,3%) y OLS (67,4%).



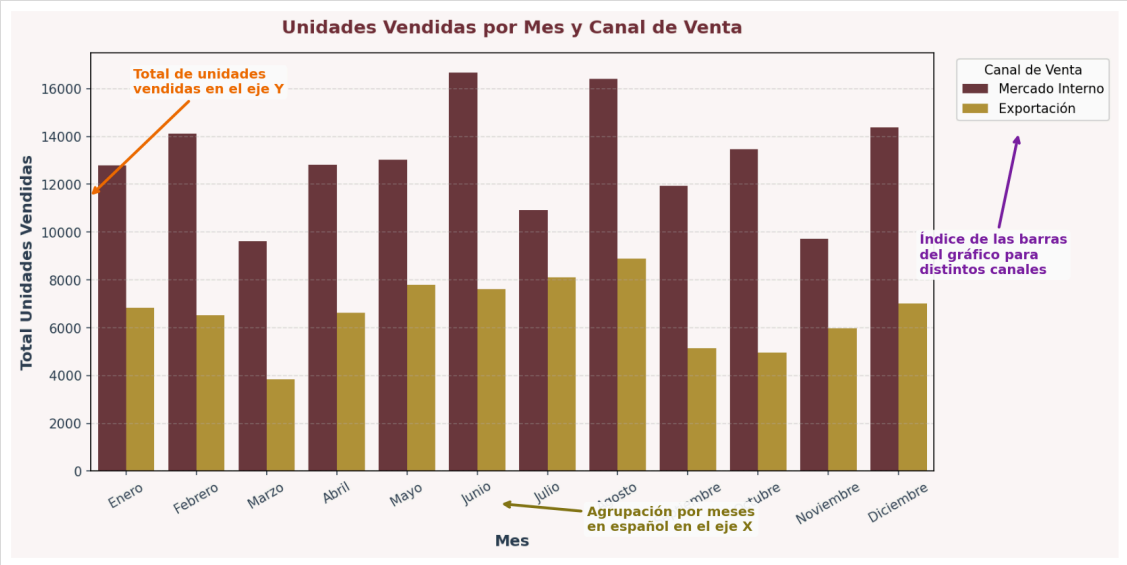
2. Gráfico de Heatmap (Matplotlib)

Este gráfico de calor indica la estacionalidad de cada varietal (eje Y) en cada estación del año (eje X), representándose con un color bordó más oscuro mientras mayor es la suma de unidades vendidas (por ejemplo, el pico de *Torrontés* en verano con 19.903 botellas y de *Cabernet Sauvignon* en invierno con 22.818). A la derecha se encuentra la barra de color que sirve de índice de escala.



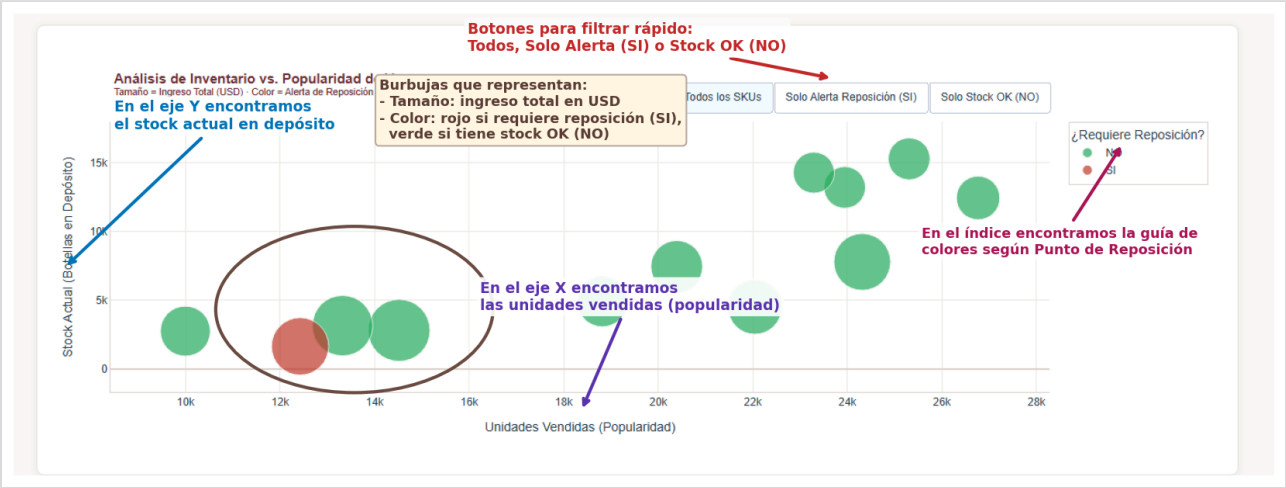
3. Gráfico de Barras Agrupadas (Seaborn)

Este gráfico de columnas agrupadas compara mes a mes (en el eje X, en español) la cantidad total de unidades vendidas (eje Y) entre el *Mercado Interno* y la *Exportación*, con su cuadro de índice arriba a la derecha.



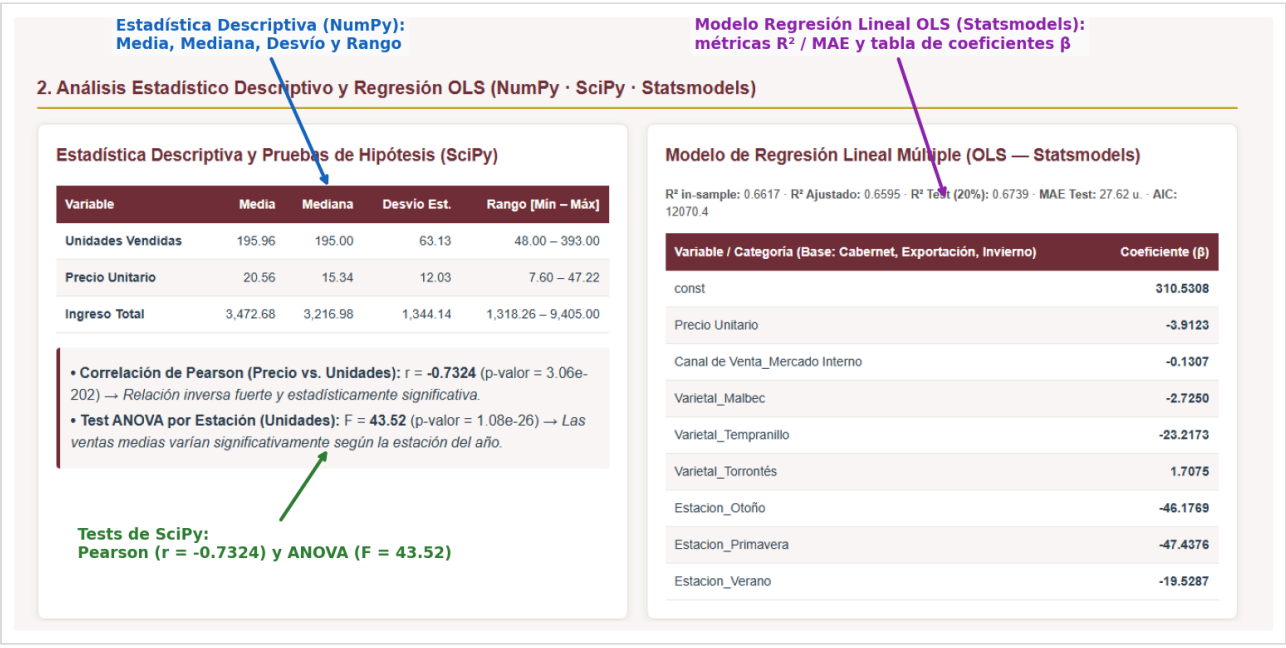
4. Gráfico de Burbujas interactivo (Plotly)

En este gráfico combinamos el inventario con el resumen de ventas y las predicciones de los dos modelos. En el eje Y tenemos el stock actual en depósito y en el eje X la popularidad (unidades vendidas). El tamaño de cada burbuja representa el ingreso total en dólares y el color indica si requiere reposición (rojo = SI, verde = NO). Además de ver todo el detalle pasando el mouse por encima, arriba a la derecha se agregaron tres botones para filtrar con un clic: *Todos los SKUs*, *Solo Alerta Reposición (SI)* o *Solo Stock OK (NO)*.



5. Panel de Estadística Descriptiva y Regresión OLS (NumPy, SciPy y Statsmodels)

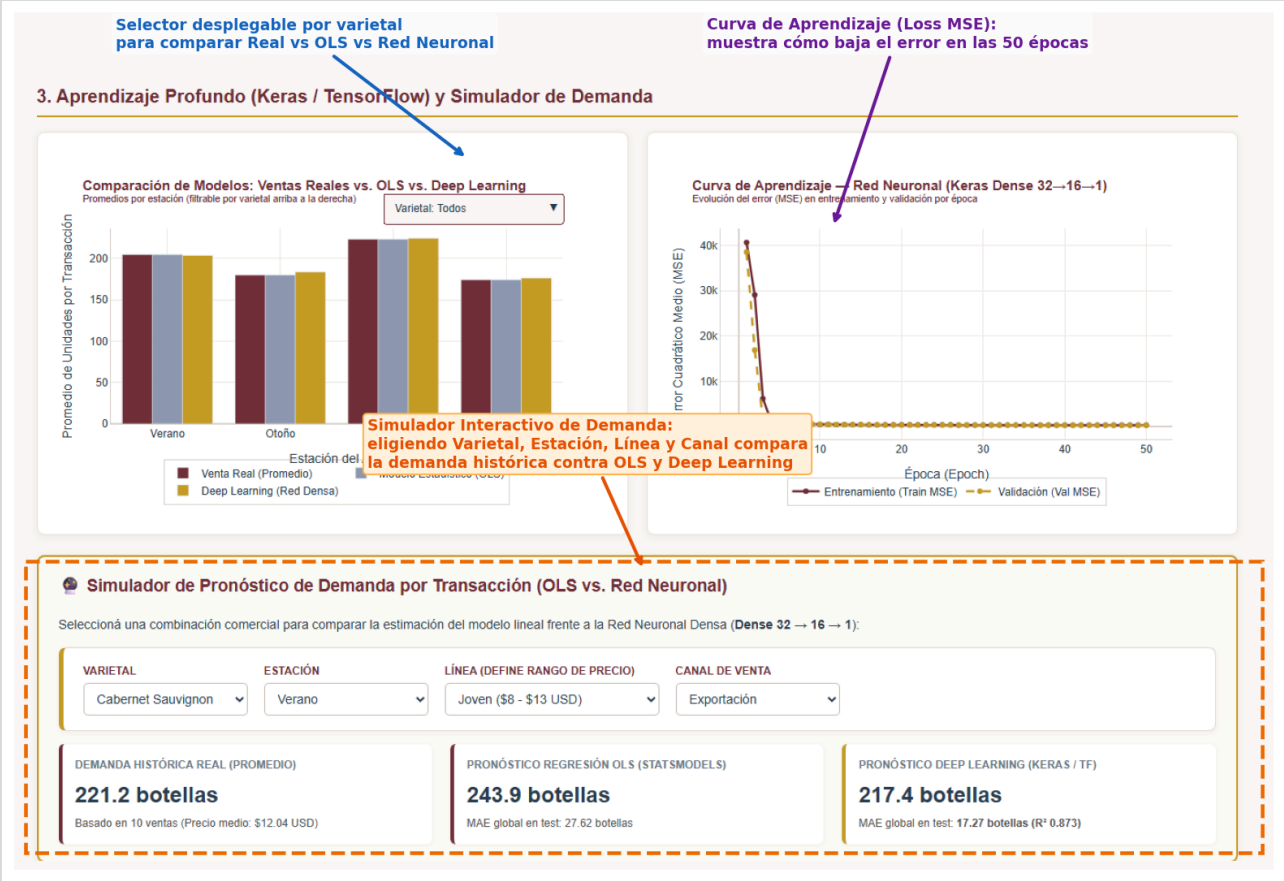
En la segunda sección del dashboard se muestran lado a lado: a la izquierda, la tabla con la Media, Mediana, Desvío Estándar y Rango junto a los resultados de la Correlación de Pearson y el test ANOVA de SciPy; y a la derecha, las métricas y la tabla con todos los coeficientes β calculados por la Regresión Lineal Múltiple OLS.



6. Panel de Aprendizaje Profundo y Simulador Interactivo de Demanda

En la tercera sección tenemos a la izquierda el gráfico interactivo de barras de Plotly que compara la *Venta Real vs. OLS vs. Red Neuronal* en cada estación (con un menú desplegable arriba

a la derecha para elegir el varietal y ver cómo la Red Neuronal sigue los picos reales); a la derecha la *Curva de Aprendizaje (MSE)* a lo largo de las 50 épocas; y abajo el **Simulador de Pronóstico de Demanda**, donde eligiendo cualquier combinación de *Varietal, Estación, Línea y Canal* se comparan en vivo las botellas promedio reales contra el pronóstico del OLS y el de la Red Neuronal.



VI. Enlace al repositorio de Github

El enlace al repositorio público del proyecto con todo el código fuente es: https://github.com/DarkosCode/WineLytics-V2

GOBIERNO DE DATOS

I. Fuentes de datos

El proyecto no utiliza datos reales de una bodega, sino datos sintéticos generados mediante un script de Python. La generación está centralizada en **sources/generadorDeDatos.py** y produce dos archivos CSV dentro de la carpeta **data/** (si no existen al clonar el repositorio, `main.py` los crea automáticamente mediante `asegurarDatos()`):

`inventario.csv`: contiene el stock actual de cada combinación Varietal + Línea (4 varietales × 3 líneas = 12 registros). Los valores de stock y punto de reposición siguen una regla de negocio simulada: las líneas más premium (Gran Reserva) tienen menos stock físico y un punto de reposición de 2500 botellas porque requieren más meses de crianza antes de poder reponerse, frente a las 5000 de la línea Joven.

`ventas.csv`: contiene 1200 transacciones de venta simuladas a lo largo del año 2025, con fecha, varietal, línea, canal de venta (Mercado Interno o Exportación), unidades vendidas y precio unitario. La generación incluye relación inversa entre precio y cantidad, estacionalidad mendocina simulada y una distribución de canal de 60% mercado interno / 40% exportación. **La semilla aleatoria está fijada (`np.random.seed(42)`)** para que los datos sean siempre exactamente los mismos entre ejecuciones.

II. Tuberías de limpieza y transformación (*pandas*)

El proceso de transformación está dividido en cuatro etapas separadas en distintos archivos:

Carga y limpieza de inventario (`sources/inventarioCsv.py`): lee `inventario.csv`, renombra las columnas originales del CSV a los nombres definitivos definidos en `schema/inventario.py`, convierte las columnas de stock y punto de reposición a entero rellenando nulos con 0 (`fillna(0).astype(int)`), elimina duplicados por `[Varietal, Linea]`, valida que el stock no sea negativo y agrega la columna calculada `Reposicion Necesaria` (**"SI"/"NO"**) comparando el stock actual contra el punto de reposición con `np.where`.

Carga y limpieza de ventas (`sources/ventasCsv.py`): lee `ventas.csv`, renombra columnas según `schema/ventas.py`, limpia nulos tipando unidades vendidas como entero y precio unitario como float, elimina filas duplicadas (`drop_duplicates()`), filtra que unidades y precios sean mayores a 0, y agrega tres columnas calculadas: `Ingreso Total` (unidades × precio), `Mes` (traducido al español de Enero a Diciembre a partir de la fecha) y `Estacion` (calculada con el método compartido `Ventas.obtenerEstacion()` según el hemisferio sur).

Cálculo de pronósticos del backend (`backend/statsAnalysis.py` y `backend/deepLearning.py`): reciben el DataFrame de ventas limpio, entrenan la Regresión Lineal OLS y la Red Neuronal Densa, y le agregan las columnas `Prediccion_Ventas` y `Prediccion_DL`.

Consolidación (`dataProcessing.py`): agrupa el DataFrame de ventas por Varietal y Línea sumando `Unidades Vendidas`, `Ingreso Total`, `Prediccion_Ventas` y `Prediccion_DL` (`groupby().sum()`), y cruza ese resumen con el inventario mediante un `pd.merge` tipo `inner` sobre las claves Varietal y Línea.

III. Diccionario de datos (*DataFrames finales*)

Se documentan los **tres DataFrames** que produce el procesamiento y que alimentan al dashboard.

DataFrame `ventas`:

Columna	Tipo	Descripción
`Fecha`	*datetime*	Fecha de la transacción (año 2025)
`Varietal`	*texto*	Varietal del vino (Malbec, Cabernet Sauvignon, Tempranillo, Torrontés)
`Linea`	*texto*	Línea del vino (Joven, Reserva, Gran Reserva)
`Canal de Venta`	*texto*	Canal de venta (Mercado Interno, Exportación)

Columna	Tipo	Descripción
`Unidades Vendidas`	*entero*	Cantidad de botellas vendidas en la transacción (variable objetivo *y*)
`Precio Unitario`	*decimal*	Precio por botella en USD equivalente
`Ingreso Total`	*decimal*	Calculada: Unidades Vendidas × Precio Unitario
`Mes`	*texto*	Calculada: nombre del mes en español (Enero a Diciembre, derivado de Fecha)
`Estacion`	*texto*	Calculada: estación del año según hemisferio sur (Verano, Otoño, Invierno, Primavera)
`Prediccion_Ventas`	*decimal*	Calculada en backend: unidades pronosticadas por la Regresión Lineal OLS
`Prediccion_DL`	*decimal*	Calculada en backend: unidades pronosticadas por la Red Neuronal (Deep Learning)

DataFrame `inventario`:

Columna	Tipo	Descripción
`Varietal`	*texto*	Varietal del vino
`Linea`	*texto*	Línea del vino
`Stock Actual`	*entero*	Unidades disponibles en depósito
`Punto de reposición`	*entero*	Umbral mínimo de stock antes de requerir reposición (2500, 3500 o 5000)
`Reposicion Necesaria`	*texto*	Calculada: "SI" o "NO" según si Stock Actual < Punto de reposición

DataFrame `consolidado` (**"combinación" de los dos**):

Columna	Tipo	Descripción
`Varietal`	*texto*	Varietal del vino
`Linea`	*texto*	Línea del vino
`Stock Actual`	*entero*	Heredada de inventario
`Punto de reposición`	*entero*	Heredada de inventario

Columna	Tipo	Descripción
`Reposicion Necesaria`	*texto*	Heredada de inventario
`Unidades Vendidas`	*entero*	Heredada de ventas, sumada por Varietal + Línea
`Ingreso Total`	*decimal*	Heredada de ventas, sumada por Varietal + Línea
`Prediccion_Ventas`	*decimal*	Heredada de ventas, suma anual estimada por la Regresión OLS
`Prediccion_DL`	*decimal*	Heredada de ventas, suma anual estimada por la Red Neuronal

GUIA DE USUARIO

I. Glosario de métricas y KPIs

Término / KPI	Definición
Ingreso Total	Métrica monetaria calculada como Unidades Vendidas × Precio Unitario. Se muestra en la primera tarjeta KPI (recalculándose con los filtros) y define el tamaño de las burbujas en el gráfico de inventario.
Unidades Vendidas (Popularidad)	Suma total de botellas vendidas (y debajo su promedio por venta). En el gráfico de burbujas se ubica en el eje X como indicador de qué tan demandado es ese vino frente al público.
Ticket Promedio	Facturación promedio en dólares por cada transacción comercial (Ingreso Total / Cantidad de transacciones), acompañada por el precio medio por botella.
Share Exportación	Porcentaje de la facturación total que proviene del canal Exportación ((Ingreso Exportación / Ingreso Total) × 100).
Stock Actual y Punto de Reposición	Cantidad de botellas disponibles en depósito (eje Y del gráfico de burbujas) comparada contra el umbral mínimo fijo de cada línea (2500 en Gran Reserva, 3500 en Reserva y 5000 en Joven).
¿Requiere Reposición? / Alerta de Stock (SKUs)	Indicador booleano ("SI"/"NO") cuando Stock Actual < Punto de Reposición. Se visualiza con color en el gráfico de burbujas (rojo = SI, verde = NO) y en la 5ta tarjeta KPI que cuenta cuántos de los 12 vinos están en alerta.
R^2 de los Modelos (Test 20%)	Coeficiente de determinación evaluado sobre el 20% de prueba: mide qué porcentaje de la variación de las ventas explica cada modelo (87,3% la Red Neuronal frente a 67,4% la Regresión OLS).

Término / KPI	Definición
MAE y RMSE	Error Absoluto Medio y Raíz del Error Cuadrático Medio medidos en botellas. Indican cuántas botellas se desvía en promedio cada modelo al pronosticar una venta (la Red Neuronal baja el MAE de 27,62 a 17,27 botellas).
Correlación de Pearson y Test ANOVA	Pruebas estadísticas de SciPy mostradas en la Sección 2: Pearson (r = -0,7324) mide la relación inversa entre precio y unidades vendidas; ANOVA (F = 43,52) confirma que la variación de ventas entre estaciones es significativa.

II. Guía de navegación e interactividad

A diferencia del primer parcial donde se abrían tres ventanas por separado, ahora al ejecutar `python3 main.py` se genera y se abre automáticamente en el navegador un único dashboard integrado en `visuals/reporte_interactivo.html`:

Barra de Filtros Globales (arriba): permite elegir un *Varietal*, una *Estación* o un *Canal de Venta* en los tres menús desplegables para recalcular al instante las tarjetas de KPIs sin recargar la página. Tocando el botón **Restablecer Filtros** se vuelve al total de las 1200 ventas.

Gráfico 1, burbujas de inventario vs. ventas (Plotly): al pasar el cursor sobre una burbuja muestra el detalle completo del vino (varietal, línea, ventas reales, demanda anual estimada por OLS y por la Red Neuronal, stock actual, punto de reposición, ingreso total y si requiere reposición). Arriba a la derecha cuenta con tres botones (*Todos los SKUs*, *Solo Alerta Reposición (SI)* y *Solo Stock OK (NO)*) para filtrar rápido los vinos, además de permitir hacer zoom arrastrando con el mouse o clic en la leyenda.

Gráficos 2 y 3, mapa de calor (Matplotlib) y barras por canal (Seaborn): se guardan automáticamente como imágenes PNG en la carpeta `visuals/` y quedan incrustados directamente dentro del dashboard debajo del gráfico de burbujas, sin bloquear la ejecución del programa.

Gráficos de Deep Learning y Simulador de Demanda (abajo): en el gráfico comparativo de modelos se puede usar el menú desplegable superior derecho para elegir un varietal puntual y comparar las barras de *Venta Real vs. OLS vs. Red Neuronal* en cada estación, junto a la curva de aprendizaje MSE a la derecha. Más abajo, en el **Simulador de Pronóstico de Demanda**, seleccionando cualquier combinación de *Varietal, Estación, Línea y Canal* se actualiza en vivo la comparación entre el promedio histórico real, el pronóstico OLS y el pronóstico de la Red Neuronal.

III. FAQ

Al igual que en el primer parcial, armé esta sección con ayuda de Gemini pensando en las dudas más comunes que pueden surgir al revisar o probar esta segunda versión del proyecto:

¿Por qué ahora se abre un solo dashboard en el navegador en vez de las tres ventanas separadas del Parcial 1?

Porque para integrar los resultados del backend (las tablas de estadística descriptiva, la regresión OLS, los gráficos de la Red Neuronal, los filtros y el simulador) de forma cómoda en un mismo lugar, se armó un reporte HTML unificado en `visuals/plotlyUtils.py`. Así tampoco hay que ir cerrando ventanas de Matplotlib y Seaborn para que el programa avance.

¿Necesito levantar un servidor web o tener internet para ver el dashboard?

No hace falta correr ningún servidor en consola (como Flask o Dash): el archivo `visuals/reporte_interactivo.html` se abre directo con doble clic en cualquier navegador. Sí hace falta tener conexión a internet para que se dibujen los gráficos interactivos de Plotly (porque carga la librería `plotly.js` desde su CDN oficial), mientras que los filtros, los KPIs, las tablas, el simulador y las imágenes de Matplotlib y Seaborn están guardados adentro del propio archivo HTML.

¿Por qué el pronóstico de la Red Neuronal (Deep Learning) da mucho mejor que el de la Regresión Lineal OLS?

Porque en las ventas de la bodega hay una combinación fuerte entre varietal y estación: el *Torrontés* se vende mucho más en verano y los tintos (*Malbec* y *Cabernet*) en invierno. El modelo lineal OLS suma los efectos por separado y no capta esa interacción (logrando un R^2 de 67,4%), mientras que la Red Neuronal, gracias a sus capas ocultas con función de activación `ReLU`, detecta sola ese patrón y sube el R^2 a 87,3% (bajando el error promedio de 27,62 a 17,27 botellas).

¿Por qué no se incluyó la columna "Linea" en la Regresión Lineal OLS?

Porque el `Precio Unitario` ya depende directamente de la línea del vino (Joven es más barato, Reserva intermedio y Gran Reserva el más caro). Poner las dos variables juntas en la regresión generaba multicolinealidad porque explicaban lo mismo; dejando el precio unitario el modelo queda más preciso y sin variables redundantes.

¿Qué pasa si clono el repositorio y no están los archivos CSV en la carpeta data/?

Los archivos generados (`data/*.csv`, `visuals/*.png` y `visuals/*.html`) están en `.gitignore` para no subir archivos pesados o autogenerados al repositorio. Cuando ejecutás `python3 main.py`, la función `asegurarDatos()` detecta sola si faltan los CSV en `data/` y ejecuta `sources/generadorDeDatos.py` automáticamente con la semilla 42 antes de correr el análisis.